Keywords: test-time scaling • retrieval-augmented reasoning • large reasoning models • chain-of-thought

ThinkRetrieve: Retrieval-Augmented Reasoning Traces for Test-Time Scaling

Dynamically Injecting Solved Exemplars Mid-Reasoning Counters Error Compounding and Improves Accuracy Across Five Models and Four Benchmarks

By Vaibhav Singh, Soumya Suvra Ghosal, Sarvesh Gharat, Soumyabrata Pal, Ramasuri Narayanam, Dinesh Manocha

arXiv:2608.10928 • Preprint, August 2026 • IIT Bombay; Univ. of Maryland; Adobe Research

Sequential test-time scaling of Large Reasoning Models (LRMs) often yields diminishing returns: longer reasoning traces compound errors and drift from the original problem. ThinkRetrieve fixes this by dynamically retrieving solved step-by-step examples from an external corpus at each intermediate reasoning step, injecting guidance on *how* to reason rather than *what facts* are relevant. Experiments across five models (1.5B–8B parameters) on GSM-8K, MATH-500, AIME 2025, and SciQ show consistent accuracy gains over standard test-time scaling.

AT A GLANCE

Problem: Allocating more inference compute (longer CoT traces) often backfires due to error compounding and problem drift in Large Reasoning Models.

Why it matters: Test-time scaling is the dominant paradigm for improving LRM accuracy; fixing its failure mode is high-leverage and training-free.

Method: At each reasoning step, retrieve the top-m solved exemplars from an external corpus (conditioned on the partial trace) and inject them into the thinking trace.

Result: Consistent accuracy gains on GSM-8K, MATH-500, AIME 2025, and SciQ across five models; largest gains on hard benchmarks and small models.

Evidence: All five models (1.5B–8B) improve; gains replicated across four benchmarks spanning math and science reasoning.

The Big Picture

Large Reasoning Models (LRMs) extend the chain-of-thought paradigm to its logical limit: instead of a few reasoning steps, they generate extended thinking traces of hundreds or thousands of tokens before producing a final answer. The more inference compute allocated, the better the performance — in principle. In practice, sequential test-time scaling saturates and then degrades: beyond some trace length, accuracy falls because the model loses track of the problem, makes errors it cannot catch, and compounds small mistakes into large final errors.

ThinkRetrieve reframes test-time scaling as a *guided* process. Rather than letting the model reason in isolation, it provides the model with solved analogues — problems whose solutions are structurally similar to the current reasoning step — at each point where the trace could diverge. The key insight is that retrieval is most useful when it is *dynamic*: the right exemplar at step 3 of a reasoning trace is different from the right exemplar at step 10.

Why Existing Approaches Fall Short

Standard RAG retrieves documents before generation and injects them once into the context. For reasoning tasks, this static approach misses the key failure mode: errors do not arise from factual ignorance but from *procedural* mistakes — applying the wrong operation, misidentifying the relevant sub-problem, or making an algebraic slip mid-trace. A retrieved document about a related topic does not provide procedural guidance.

Best-of-N sampling addresses error compounding by generating N independent traces and selecting the best one, but is expensive (linear in N) and does not actually fix any individual trace — it relies on at least one trace avoiding the failure mode. ThinkRetrieve fixes the trace *at the point of failure*.

Core Mechanism

ThinkRetrieve wraps the LRM's generation loop with a retrieval step at each reasoning checkpoint:

1. At reasoning step k, form a query $q_k = [\text{problem}; \text{partial trace}_k]$.
2. Retrieve top-m solved exemplars (p_j, sol_j) from corpus $\mathcal{C}$ based on embedding similarity to q_k.
3. Inject the exemplars as additional in-context examples preceding step k.
4. Continue the LRM's generation with the augmented context.

The corpus $\mathcal{C}$ is a collection of (problem, step-by-step solution) pairs; no fine-tuning of the LRM is performed. The retrieval encoder is a standard dense retriever (e.g., a contrastively trained bi-encoder).

Technical Formulation

Let π_θ be the LRM policy and x the input problem. Standard decoding generates the trace $t = (t_1, \ldots, t_L)$ autore-

gressively:

$$p(t \mid x) = \prod_{k=1}^{L} p_\theta(t_k \mid x, t_{<k})$$

ThinkRetrieve modifies the conditioning at each step by inserting retrieved exemplars $\mathcal{E}_k = \text{Retrieve}(\mathcal{C}, [x; t_{<k}])$:

$$p^{\text{TR}}(t_k \mid x, t_{<k}) = p_\theta(t_k \mid \mathcal{E}_k, x, t_{<k})$$

where $\mathcal{E}_k$ is updated at each reasoning checkpoint (every δ tokens). The final answer is extracted from t_L as in standard decoding.

Retrieval similarity is cosine distance in the embedding space:

$$\text{sim}(q_k, p_j) = \frac{\text{enc}(q_k)^\top \text{enc}(p_j)}{\|\text{enc}(q_k)\| \cdot \|\text{enc}(p_j)\|}$$

Top-m exemplars by similarity are selected and prepended to the context.

Learning or Inference Procedure

No training is involved. The LRM (π_θ) and dense retriever are both frozen. The only hyperparameters are:

- m: number of retrieved exemplars per step (ablated: 1, 3, 5)
- δ: retrieval frequency (every δ generated tokens)
- Retrieval encoder: the paper uses a standard off-the-shelf bi-encoder

What the Guarantee Says

No formal guarantee is provided. The empirical claim is that dynamic step-conditioned retrieval consistently improves accuracy over static retrieval and over no retrieval, across model scales from 1.5B to 8B parameters and across four diverse benchmarks. The consistency across scales and tasks provides strong evidence that the mechanism is general rather than benchmark-specific.

Experimental Findings

- **GSM-8K and MATH-500:** Consistent accuracy gains across all five models; gains increase with trace length (longer traces benefit most).
- **AIME 2025:** Largest absolute gains, where baseline traces are most error-prone and problem drift is most severe.
- **SciQ:** Gains confirmed on a non-math reasoning benchmark, establishing cross-domain generality.
- **Small models (1.5B–3B):** Largest relative gains; ThinkRetrieve provides the most leverage where baseline reasoning is weakest.
- **Ablation on m:** $m = 3$ is near-optimal; $m = 1$ is 60–80% of the gain, $m > 5$ provides marginal additional benefit at higher context cost.

Ablations and Interpretation

Dynamic vs. static retrieval: Retrieving once at the start (standard RAG) provides ∼30% of the accuracy gain of dynamic per-step retrieval — confirming that the value is in the conditioning on the *partial trace*, not just the problem.

Retrieval timing: Injecting exemplars early in the trace (step 1–2) is most helpful for math benchmarks, where problem setup is the hardest phase; mid-trace retrieval dominates for multi-step science problems.

Corpus quality: Using a corpus of correct solutions consistently outperforms a corpus of incorrect-but-plausible solutions, confirming that the model is learning procedural structure rather than just surface similarity.

Reference

Vaibhav Singh, Soumya Suvra Ghosal, Sarvesh Gharat, Soumyabrata Pal, Ramasuri Narayanam, Dinesh Manocha. **ThinkRetrieve: Retrieval-Augmented Reasoning Traces for Test-Time Scaling**. arXiv:2608.10928, August 2026. https://arxiv.org/abs/2608.10928